Beyond IDs: How Emergent Identity Solves Entity Resolution Without Rules

Antonio Fallea
antoniofallea2005@gmail.com
github.com/antofallea

July 2026

Abstract

Entity Resolution (ER) traditionally requires handcrafted rules, labeled training data, or probabilistic models with explicit optimization objectives. This paper introduces the **Relational Identity Structure (RIS)**, a procedural data structure that computes entity identity from relationship topology through iterative signature computation, thresholded similarity, and irreversible node merging. RIS produces a *set* of possible partitions (one per valid merge schedule) rather than a unique optimum.

We provide: (1) a well-posedness proof establishing termination, finite output, and computability for all admissible inputs; (2) formal characterization as a terminating, non-confluent, state-dependent reduction process with locally-bounded updates ($O(d_{\text{avg}}^2)$ affected nodes per edge change); (3) an explicit analysis of merge non-monotonicity, showing that while counterexamples exist in the ambient embedding space, empirical non-monotonic behavior occurs in $< 0.3\%$ of merges; (4) set-valued output semantics that make the system's schedule-dependence explicit and measurable; and (5) experimental results on three benchmarks achieving 95.3% F1 without domain-specific matching rules, competitive with supervised methods requiring thousands of labeled examples.

We emphasize that RIS is a **procedural algorithm**, not an optimizer: it has no internal objective function, and evaluation metrics are external quality probes. RIS defines a transformation dynamics on graphs; any interpretation in terms of clustering or identity is external to the formal system. Approximately 60% of RIS's dynamic gain over static clustering comes from conflict resolution and temporal tracking; 40% from transitive closure. We release the complete implementation, benchmarks, and reproduction scripts as open source.

Keywords: entity resolution, dynamical clustering, locally-bounded updates, procedural identity, set-valued semantics, state-dependent reduction

[0]**Positioning and Scope.** This paper describes RIS, a procedural algorithm for entity resolution based on iterative signature computation and thresholded merging.

Contributions: (1) An operational framework where identity is computed from relational structure rather than assigned via static keys. (2) Formal characterization of the algorithm's properties: termination (global), non-uniqueness of terminal graphs (global), state-dependent reduction, schedule-parametric determinism (per-policy), and locally-bounded updates. (3) Explicit documentation of all limitations: no optimality guarantees, schedule sensitivity, empirical rather than theoretical convergence, and absence of domain-independent threshold selection. (4) Competitive empirical performance on three benchmarks without domain-specific matching rules.

Non-claims: This paper does *not* claim that RIS is optimal, correct in any formal sense, unique-output, or equivalent to any existing formal system (ARS, graph rewriting, optimization). It claims only that this specific combination of design choices and formal properties is novel in the entity resolution literature and practically useful.

Theoretical framing: We use basic rewriting terminology (reduction step, terminal graph, termination) for descriptive clarity. We do *not* rely on, apply, or claim results from standard rewriting theory (confluence, Church-Rosser, unique normal forms), which assumes a static reduction relation not present in RIS.

Epistemic separation: RIS defines a transformation dynamics on graphs; any interpretation of the output in terms of "clustering" or "identity" is external to the formal system. The system specifies what transformations occur and under what conditions; it does not define what those transformations *mean*. This separation is essential for understanding the scope of our claims.

1 Introduction

1.1 The Identity Assumption

Every database, every graph, every knowledge system relies on the same fundamental assumption: **entities have permanent IDs**. A user is `user_12345`. A product is `product_67890`. These IDs are assigned at creation and never change, regardless of what the entity represents or how it evolves.

This assumption creates a fundamental tension: real-world entities change, merge, split, and evolve, but our data structures treat them as immutable atoms. When two records represent the same real-world entity but carry different IDs, we face the Entity Resolution problem—a challenge that costs organizations millions annually in data quality efforts [1].

1.2 Contributions

This paper introduces **Relational Identity Structure (RIS)**, a data structure that inverts the traditional identity assumption. Instead of treating identity as a label assigned at creation, RIS computes identity continuously from an entity's relational context. When two entities exhibit structurally equivalent relationships, they automatically unify—eliminating the need for explicit deduplication rules.

Our key contributions are:

1. **A formal model** of identity as a computable function of relational topology (Section 3)
2. **An embedding-based framework** that represents entities as vectors derived from their relationships, with formal complexity bounds (Section 3.5)
3. **Automatic merging** based on structural equivalence without rule engineering or training data (Section 3.4)
4. **Continuous identity tracking** that reflects real-world entity evolution, with formal characterization of order-dependence and per-policy determinism (Section 3.6)
5. **Rigorous experimental validation** on three standard benchmarks with multiple baselines, including unsupervised methods and fair comparison protocols (Section 5)
6. **Open-source implementation** with complete reproduction package

Scope and modesty of claims. This paper describes a system and characterizes its properties. We do not prove optimality, correctness, or expressiveness theorems. The value proposition is practical (entity resolution without domain-specific rules, competitive with supervised methods) and conceptual (identity as a procedural, relation-derived notion). Readers seeking theoretical guarantees of correctness or optimality should consult the probabilistic record linkage literature; RIS is a complementary approach for settings where such guarantees are unavailable or unnecessary.

Epistemic separation. RIS defines a transformation dynamics on graphs; any interpretation of the output in terms of "clustering" or "identity" is external to the formal system. The system specifies what transformations occur and under what conditions; it does not define what those transformations *mean*. This separation is essential for understanding the scope of our claims throughout the paper.

1.3 Paper Structure

Section 2 surveys related work. Section 3 formalizes the RIS model. Section 4 describes the implementation. Section 5 presents experimental results. Section 6 discusses limitations and future work. Section 7 concludes.

2 Related Work

2.1 Traditional Entity Resolution

Entity Resolution (ER), also known as Record Linkage or Deduplication, has been studied extensively since the foundational work of Newcombe et al. [10] and Fellegi & Sunter [2]. Traditional approaches fall into several categories:

Rule-Based Methods rely on handcrafted matching logic. Systems like Dedupe [12] require domain experts to define blocking keys and similarity thresholds. While interpretable, these methods scale poorly with data diversity—new data sources require new rules, creating a maintenance burden that grows quadratically with schema complexity.

Probabilistic Record Linkage extends the Fellegi-Sunter framework using likelihood ratios for field-level comparisons. Modern implementations [13] incorporate Bayesian priors and EM estimation for parameter learning. However, these methods require careful feature engineering and assume field independence—an assumption often violated in practice.

Machine Learning Approaches train classifiers on labeled pairs. DeepMatcher [9] uses RNNs with attention to learn similarity functions from training data. Ditto [6] leverages pre-trained language models fine-tuned on ER tasks. While achieving state-of-the-art results, these methods require expensive labeled datasets containing both matches and non-matches, and models must be retrained as data distributions shift.

2.2 Graph-Based Identity

Graph embeddings provide vector representations of nodes based on structural properties. Node2Vec [3] learns embeddings through biased random walks. GraphSAGE [4] uses neural networks to aggregate features from node neighborhoods. Graph Convolutional Networks [5] propagate information through spectral convolutions.

These methods compute **similarity** between nodes but do not treat identity as emergent. Two nodes with similar embeddings remain distinct entities unless explicitly merged through external logic. RIS builds upon graph embedding techniques but introduces a critical distinction: **identity becomes a first-class property of the relational structure**, not just a similarity score.

2.3 Structural Equivalence Theory

Lorrain & White [7] introduced structural equivalence in social network analysis: two actors are structurally equivalent if they have identical relationships to all other actors. White & Reitz [11] extended this to regular equivalence. RIS operationalizes structural equivalence by making it the basis for identity computation, treating equivalence as grounds for unification rather than mere classification.

2.4 Relationship to Formal Systems

Abstract State Machines (ASMs). Gurevich's ASMs [15] provide a framework for specifying algorithms at arbitrary levels of abstraction. RIS can be viewed as an ASM where the state is the graph G, and the transition rule is: "if $\exists(a, b) : \text{sim}_G(a, b) \geq \tau$, then $G := \mathcal{M}(G, a, b)$." However, standard ASM theory assumes that transition rules are fixed, while in RIS the *applicability condition* ($\text{sim}_G \geq \tau$) depends on a function computed from the state. This self-referential aspect distinguishes RIS from standard ASMs.

Graph Rewriting Systems. The algebraic approach to graph transformation [16] models system evolution through rule-based graph modifications. RIS is a special case where: (1) the only transformation is node merging, (2) the applicability condition is computed via embedding similarity rather than pattern matching, and (3) the condition itself evolves with the graph. This combination of properties is not captured by standard graph rewriting formalisms.

This paper's relationship to formal systems: RIS occupies an under-explored region in the space of formal systems—between ASM and graph rewriting—characterized by state-dependent applicability conditions derived from learned embeddings. We use basic rewriting terminology for descriptive clarity only, without relying on or claiming results from standard rewriting theory.

2.5 Gap Analysis

No existing system provides:

- Identity as a continuous, computed property rather than a static label
- Automatic merging without external rules or training data
- Real-time identity updates as relationships change
- Explicit characterization of schedule-dependence with set-valued semantics
- Formal analysis as a state-dependent reduction process with locally-bounded updates

RIS addresses this gap by unifying structural equivalence theory with modern embedding techniques in a single operational data structure with formal guarantees on termination and update locality.

3 The Relational Identity Structure Model

Notational Convention: Policy-Dependent Properties

Throughout this section, we distinguish three levels of properties:

1. **Per-schedule properties:** Hold for a specific reduction sequence (e.g., number of merge steps in σ)
2. **Per-policy properties:** Hold for all schedules generated by a fixed policy π (e.g., terminal graph reached by π_{batch})
3. **Global properties:** Hold for all admissible schedules and policies (e.g., termination, bound on $|V|$ reduction)

When a property depends on the schedule or policy, we indicate this explicitly. Properties stated without qualification are global.

3.1 Formal Definition

Definition 3.1 (RIS Graph)**.** *An RIS instance is a 4-tuple $G = (V, E, L, w)$ where:*

- *V is the set of nodes*
- *$E \subseteq V \times V$ is the set of directed edges*
- *L is the set of relation labels*
- *$w : E \to \mathbb{R}^+$ is the weight function*

Each edge $e = (u, v) \in E$ carries a label $\lambda \in L$ and weight $w(e)$.

Definition 3.2 (Relational Neighborhood)**.** *For node $v \in V$, its relational neighborhood is:*

$$N(v) = \{(u, \lambda, w) \mid (v, u) \in E \vee (u, v) \in E\} \tag{1}$$

Unlike traditional databases, nodes do **not** carry intrinsic identity attributes. Their identity is determined entirely by $N(v)$.

3.2 Relational Signature

Definition 3.3 (Signature Function)**.** *A signature function $\phi : V \to \mathbb{R}^d$ maps each node to a d-dimensional vector:*

$$\phi(v) = \textit{normalize}\left(\sum_{(u,\lambda,w)\in N(v)} w \cdot \textit{encode}(\lambda, u)\right) \tag{2}$$

where encode : $L \times V \to \mathbb{R}^d$ is an embedding function and normalize ensures unit ℓ_2 norm.

Implementation Choices:

- **Hash-based:** $\text{encode}(\lambda, u) = \text{hash}(\lambda) \oplus \text{hash}(u)$ projected to $\mathbb{R}^d$ via random projection
- **Learned:** $\text{encode}(\lambda, u) = W[\lambda] \cdot \text{embed}(u)$ where W is a learned tensor
- **Pretrained:** $\text{encode}(\lambda, u) = \text{GNN}(v)$ using a pretrained Graph Neural Network

The signature $\phi(v)$ is a relational fingerprint: nodes with identical relationships produce identical signatures.

3.3 Induced Structure of Signature Space

The signature function ϕ is not surjective onto $\mathbb{R}^d$. Its image $\Phi(G)$ is constrained by:

1. **Finite encoding alphabet:** The hash-based encoding produces vectors from a discrete set of size $|L| \times |V|$. The signature is a weighted sum from this finite dictionary.
2. **Non-negative combinations:** Since weights $w \geq 0$, signatures lie in the convex cone generated by the encoding vectors of connected neighbors.
3. **Normalization to unit sphere:** Normalization projects onto $\mathbb{S}^{d-1}$.
4. **Structural constraints:** Not all vectors on $\mathbb{S}^{d-1}$ are realizable. $\Phi(G)$ is a subset of the rational points on the sphere induced by the encoding dictionary.

For hash-based encoding with $d = 64$ and a graph with $n = 10^4$ nodes and $|L| = 10$ relation types, the number of possible encoding vectors is 10^5, while the number of points on a quantized $\mathbb{S}^{63}$ is vastly larger. This suggests that $\Phi(G)$ is an **extremely sparse** subset of the embedding space, which may explain the empirical rarity of non-monotonic configurations.

3.4 Closure Under Merge

Observation 3.4 (Closure Under Merge)*.* After merging nodes a and b into a', the signature $\phi(a')$ is computed from $N(a') = N(a) \cup N(b)$. Since $N(a')$ is a valid neighborhood in the post-merge graph G', $\phi(a')$ is a valid signature in $\Phi(G')$. The merge operator preserves membership in Φ: it maps valid signatures to valid signatures in the new graph. This is a consistency property that follows from the definition of signatures as functions of neighborhoods.

3.5 Identity Similarity

Definition 3.5 (Identity Similarity)**.** *The similarity between nodes a and b is:*

$$\textit{sim}(a, b) = \frac{\phi(a) \cdot \phi(b)}{\|\phi(a)\| \cdot \|\phi(b)\|} \tag{3}$$

This cosine similarity ranges from -1 (opposite relational profiles) to 1 (identical relational profiles). In practice, since all weights are non-negative, the range is $[0, 1]$.

Interpretation Guide:

Similarity Range	Interpretation	Action
$[0.98, 1.00]$	Structurally identical	Automatic merge
$[0.90, 0.98)$	Highly similar	Candidate for manual review
$[0.70, 0.90)$	Related entities	Potential linkage
$[0.00, 0.70)$	Distinct entities	No action

3.6 Merge Criterion

Definition 3.6 (Merge Operation)**.** *Two nodes $a, b \in V$ are merged when $sim(a, b) \geq \tau$, where $\tau \in [0, 1]$ is the merge threshold. The merge operation $\mathcal{M} : G \to G'$ is:*

$$V' = V \setminus \{b\} \tag{4}$$

$$E' = \{(x, y) \in E \mid x \neq b \wedge y \neq b\} \cup \{(a, y, \lambda, w) \mid (b, y, \lambda, w) \in E\} \tag{5}$$

$$alias[b] = a \tag{6}$$

Node b becomes an alias of a, and all of b's relationships transfer to a. External references to b remain valid through the alias table.

3.7 RIS as a Dynamical System

The interaction between signature computation and merging creates a feedback loop formalized as a discrete-time dynamical system.

Definition 3.7 (RIS State and Transition)**.** *The state of an RIS at time t is the graph $G^{(t)} = (V^{(t)}, E^{(t)}, L, w)$. The transition operator $\mathcal{T} : \mathcal{G} \to \mathcal{G}$ is defined as:*

$$\mathcal{T}(G) = \mathcal{M}^*(G, \Phi(G)) \tag{7}$$

where $\Phi(G)$ computes all pairwise similarities and $\mathcal{M}^*$ applies all merges exceeding threshold τ.

Definition 3.8 (Fixed Point)**.** *A graph G^* is a **terminal graph** (fixed point) of RIS if no pair (a, b) satisfies $sim_{G^*}(a, b) \geq \tau$.*

Theorem 3.9 (Convergence under Batch Merging — Global Property)**.** *Under batch merging, the sequence $G^{(0)}, G^{(1)}, G^{(2)}, \ldots$ reaches a terminal graph in at most $|V^{(0)}|$ iterations.*

Proof. Each merge reduces $|V|$ by at least 1. Since $|V|$ is finite and non-negative, the process must terminate. The terminal state G^* satisfies $\mathcal{T}(G^*) = G^*$ by definition. □

Proposition 3.10 (Batch Merge Idempotence — Per-Policy Property)**.** *For the batch policy π_{batch}, one pass of batch merging is sufficient to reach the terminal graph: $\mathcal{T}(\mathcal{T}(G)) = \mathcal{T}(G)$.*

Proof. Batch merging computes all pairwise similarities on the current graph state before any merges occur. After merging, nodes only gain relationships (from merged nodes), which can only increase similarity. Since all pairs exceeding τ were already merged in the first pass, no new pairs exceed τ in the second pass. □

3.8 Sequential Mode and Order Dependence

Proposition 3.11 (Sequential Order Dependence — Global Property)**.** *In sequential mode, the terminal graph depends on merge order. Different reduction sequences from the same initial graph may reach different terminal graphs.*

Construction. Let A, B, C be nodes where $\text{sim}(A, B) \geq \tau$, $\text{sim}(B, C) \geq \tau$, but $\text{sim}(A, C) < \tau$ initially. If we merge $A \to B$ first, B's signature changes by absorbing A's relationships. This may increase $\text{sim}(B, C)$ above τ, causing a subsequent merge. If we merge $B \to C$ first, the same process may lead to a different terminal graph. □

Mitigation Strategies:

1. **Batch mode (recommended):** Compute all pairwise similarities before any merges. Eliminates order dependence at cost $O(n^2)$ similarity computations per batch.
2. **Deterministic ordering:** Process merges by decreasing similarity score. Reduces variance but does not eliminate it.
3. **Multi-pass until fixed point:** Apply sequential merges repeatedly until terminal graph is reached. Theorem 3.9 guarantees termination.
4. **Conflict resolution:** When node A can merge with both B and C, choose the highest-similarity pair.

3.9 On the Non-Monotonicity of Merge Operations

We provide a rigorous analysis of merge monotonicity, correcting an earlier claim.

Proposition 3.12 (Merge Operations Are Not Monotonic in General — Global Property)**.** *There exist graphs G and nodes a, b, c such that after merging a and b into a', $sim(a', c) < \max(sim(a, c), sim(b, c))$. This counterexample exists in the space of all possible signature vectors $\mathbb{R}^d$.*

Constructive counterexample. Let $d = 2$. Consider:

$$\phi(a) = (1, 0) \tag{8}$$

$$\phi(b) = (-0.5, 0.866) \quad (120^\circ \text{ from a}) \tag{9}$$

$$\phi(c) = (0.866, 0.5) \tag{10}$$

Then $\text{sim}(a, c) \approx 0.866$, $\text{sim}(b, c) \approx 0$. After merging, $\phi(a')$ is the bisector, and $\text{sim}(a', c)$ can be strictly less than $\text{sim}(a, c)$. □

Remark 3.13 (Restriction to Graph-Induced Signatures). The above counterexample uses arbitrary vectors in $\mathbb{R}^d$. However, RIS signatures are not arbitrary: they are constrained to the **induced manifold** $\Phi(G)$. It is an open question whether non-monotonicity occurs *within* $\Phi(G)$. Empirically, we observe non-monotonic behavior in $< 0.3\%$ of merge operations on our benchmarks, suggesting that either the induced manifold rarely contains adversarial configurations, or that the encoding function's properties make counterexamples unlikely. We leave the theoretical question to future work.

Observation 3.14 (Empirical Near-Monotonicity). Despite the theoretical counterexample, similarity decreases occur in $< 0.3\%$ of merge operations on our benchmark datasets. This is because: (1) signatures are high-dimensional ($d \geq 64$), making orthogonal additions unlikely; (2) merged nodes already have high similarity, so their signature vectors are in a narrow cone; (3) real-world relationship graphs rarely contain the adversarial structures needed for counterexamples. This is an **empirical observation**, not a guarantee.

Implications for theory:

- Convergence guarantees (Theorem 3.9) rely only on finite state space and decreasing $|V|$, not monotonicity—they remain valid
- The system can theoretically oscillate, though we have not observed this in practice
- Non-monotonicity is documented as a property of the system, not a bug to be fixed

3.10 Computational Complexity

Theorem 3.15 (Complexity Bounds — Global Property)**.** *Let $n = |V|$, $m = |E|$, and d_{avg} be the average node degree. Then:*

- *Insert node: $O(1)$*
- *Add/remove edge: $O(d_{avg})$ for signature update, $O(\log n)$ for index update with HNSW*

- *Identity query: $O(\log n)$ amortized with HNSW, $O(n)$ worst-case with flat index*
- *Merge: $O(d_1 + d_2)$ where d_1, d_2 are degrees of merged nodes*

Space complexity is $O(n + m + nd)$, linear in graph size and embedding dimension.

Proof. Insert creates one node record and one zero-vector: $O(1)$. Edge operations require updating signatures of the two endpoints: each signature computation iterates over $O(d_{\text{avg}})$ neighbors. HNSW insertion and query are $O(\log n)$ amortized [8]. Merge transfers edges from one node to another: $O(d_1 + d_2)$ edge updates. Space stores n node records, m edges, and n vectors of dimension d: $O(n + m + nd)$. □

3.11 Identity Strength

Definition 3.16 (Identity Strength)**.** *The strength of a node's identity is:*

$$\textit{strength}(v) = \left\| \sum_{(u,\lambda,w)\in N(v)} w \cdot \textit{encode}(\lambda, u) \right\| \tag{11}$$

A node with many strong relationships has high identity strength—it is "well-defined" in the relational space. A completely disconnected node has zero identity strength.

3.12 Update Locality

Theorem 3.17 (Update Propagation — Global Property)**.** *When an edge (v, u, λ, w) is added or removed, only v and nodes within distance 2 of v require signature recomputation.*

Proof. The signature $\phi(v)$ depends only on $N(v)$. Changing $N(v)$ affects $\phi(v)$ directly. For any neighbor $u \in N(v)$, $N(u)$ changes, requiring recomputation of $\phi(u)$. Nodes at distance > 2 from v are unaffected because neither their direct neighborhood nor the neighborhoods of their neighbors change. □

3.13 Stability Under Perturbation

Theorem 3.18 (Locality of Perturbation Impact — Global Property)**.** *Let G and G' differ by one edge (u, v). The number of identity changes (different merge decisions) induced by this perturbation is bounded by the size of the 2-hop neighborhood of $\{u, v\}$.*

Proof. Only nodes in the 2-hop neighborhood change signatures. Only merges involving these nodes can differ between runs. Nodes outside this neighborhood have identical signatures and identical similarities to each other, so their merge decisions are identical. □

Corollary 3.19 (Expected Impact Under Random Graphs)**.** *For an Erdős–Rényi random graph $G(n, p)$, the expected number of nodes affected by a single edge perturbation is $O(1)$ for sparse graphs ($p = O(1/n)$) and $O(n^2)$ for dense graphs (constant p).*

Interpretation: The locality guarantee is meaningful for **sparse graphs**, which characterize most real-world entity resolution scenarios. For dense graphs, RIS loses its locality advantage.

3.14 Semantics of RIS Output

Since the terminal graph depends on the merge schedule, we must define what RIS actually produces.

Definition 3.20 (Merge Schedule)**.** *A merge schedule σ is a sequence of merge operations $(a_1, b_1), (a_2, b_2), \ldots$ applied to an initial graph $G^{(0)}$, where each pair (a_i, b_i) satisfies $\textit{sim}_{G^{(i-1)}}(a_i, b_i) \geq \tau$.*

Definition 3.21 (Maximal Admissible Schedule)**.** *A merge schedule σ is **maximal** if it terminates only when no pair (a, b) satisfies $\textit{sim}_{G^{(t)}}(a, b) \geq \tau$. We restrict attention to maximal schedules throughout.*

Definition 3.22 (RIS Semantics — Global Property)**.** *The output of RIS on initial graph $G^{(0)}$ is a* ***set-valued functional****:*

$$\mathcal{R}(G^{(0)}) = \{\mathcal{P}_\sigma \mid \sigma \text{ is a maximal admissible merge schedule}\} \tag{12}$$

where $\mathcal{P}_\sigma$ is the partition produced by schedule σ.

Interpretation: RIS does not produce a single answer. It produces a **set of possible answers**, one for each valid merge schedule. When we report a single partition, we are selecting one representative from this set using a deterministic schedule (batch mode with fixed ordering).

Observation 3.23 (Empirical Schedule Stability). On our benchmark datasets, the F1 variance across 50 random merge schedules is $\sigma^2 = 0.0017$, and the maximum F1 difference between any two schedules is 0.023. This suggests $\mathcal{R}(G^{(0)})$ is **tightly clustered** for realistic entity resolution graphs.

3.15 Policy and Determinism

Definition 3.24 (Schedule Policy)**.** *A* ***schedule policy*** *π is a deterministic function that, given a graph G and a set of mergeable pairs $S = \{(a, b) \mid sim_G(a, b) \geq \tau\}$, selects either one pair to merge or halts.*

Proposition 3.25 (Determinism — Per-Policy Property)**.** *For a fixed policy π, RIS_π is deterministic: it produces a unique terminal graph $\mathcal{P}_\pi(G^{(0)})$ for any initial graph $G^{(0)}$.*

Proof. Policy π selects a unique next action at each step. Since the graph state and similarity function are deterministic given the history, the entire reduction sequence is uniquely determined. □

Standard policies:

- **Batch policy (π_{batch}):** Compute all pairwise similarities from current graph; merge all pairs above threshold simultaneously
- **Greedy policy (π_{greedy}):** Merge the highest-similarity pair first; recompute signatures; repeat
- **Fixed-order policy (π_{order}):** Process nodes in lexicographic order; merge with first partner above threshold

The set-valued semantics corresponds to $\mathcal{R}(G^{(0)}) = \{\mathcal{P}_\pi(G^{(0)}) \mid \pi \in \Pi\}$ where Π is the set of all admissible policies.

Remark 3.26 (Policy Choice is a Modeling Decision). The choice of policy is not a hyperparameter to be tuned, but a **modeling decision** about the semantics of identity in the application domain. Batch policy corresponds to synchronous identity resolution (all evidence considered simultaneously). Greedy policy corresponds to asynchronous resolution (strongest evidence first, cascading effects allowed).

3.16 Well-Posedness

Proposition 3.27 (Well-Posedness — Global Property)**.** *For any finite initial graph $G^{(0)} = (V^{(0)}, E^{(0)}, L, w)$, every maximal admissible merge schedule σ:*

1. ***Terminates*** *in at most $|V^{(0)}| - 1$ merge steps*
2. ***Produces a finite partition*** *$\mathcal{P}_\sigma$ of $V^{(0)}$*
3. ***Is computable*** *in $O(|V^{(0)}|^2 \cdot d_{avg})$ time for the batch schedule*

Therefore, RIS is a well-defined computational problem.

Proof. **(1)** Each merge reduces $|V|$ by exactly 1. The process cannot continue below $|V| = 1$. **(2)** After termination, surviving nodes form a partition of the original node set via the alias table. **(3)** Each merge requires signature computation and similarity search. □

3.17 Properties of the Reduction Process

We characterize the behavior of RIS using basic reduction terminology, avoiding reliance on results from rewriting theory.

Definition 3.28 (Reduction Step)**.** *A **reduction step** $G \xrightarrow{a,b} G'$ occurs when nodes a, b in graph G satisfy $sim_G(a, b) \geq \tau$, and $G' = \mathcal{M}(G, a, b)$. A **reduction sequence** is a chain $G_0 \xrightarrow{a_1,b_1} G_1 \xrightarrow{a_2,b_2} \cdots$. A **terminal graph** is a graph where no reduction step is possible.*

Theorem 3.29 (Termination — Global Property)**.** *Every reduction sequence reaches a terminal graph in finitely many steps. The number of reduction steps in any sequence from $G^{(0)}$ is bounded by $|V^{(0)}| - 1$.*

Proof. Each reduction step merges two nodes, reducing $|V|$ by exactly 1. Since $|V| \geq 1$, the process must stop after at most $|V^{(0)}| - 1$ steps. □

Theorem 3.30 (Non-Uniqueness of Terminal Graphs — Global Property)**.** *Different reduction sequences from the same initial graph may reach different terminal graphs.*

Proof. Constructive example provided in Section 3.6 (Order Dependence). □

Observation 3.31 (State-Dependent Reduction)*.* Unlike standard rewriting systems where the reduction relation is fixed a priori, the set of enabled reductions in RIS depends on the current graph via sim_G. After each merge, the similarity function changes, so the set of reducible pairs changes. This is a **state-dependent reduction process**: the reduction relation co-evolves with the state. Results from standard rewriting theory (Church-Rosser, Newman's Lemma) assume a static reduction relation and are not applicable here.

Remark 3.32 (Relationship to Graph Rewriting)*.* RIS can be described as a restricted form of graph rewriting [16] where the only production rule is node merging, and the rule's applicability condition depends on an embedding-based similarity function computed from the current graph. This specific combination is, to our knowledge, not standard in the graph rewriting literature. We use basic rewriting terminology for descriptive clarity only, without relying on or claiming results from rewriting theory.

3.18 On the Relationship Between RIS and Optimization

Observation 3.33 (RIS is Not Defined via Optimization)*.* RIS is defined procedurally (Section 3.5), not as the solution to an optimization problem. There is no objective function that RIS explicitly minimizes or maximizes. This is a **definitional fact**, not a theoretical claim about expressiveness.

Remark 3.34 (Comparison with Optimization-Based Methods)*.* Many clustering and entity resolution methods are defined as solutions to optimization problems: correlation clustering minimizes disagreements, spectral clustering minimizes normalized cut, Fellegi-Sunter maximizes posterior match probability. RIS differs in that it is defined by an iterative process (compute signatures, merge, repeat) rather than by a target function to optimize.

This does **not** mean RIS cannot be *described* as optimizing something. Any terminating deterministic process can be characterized as optimizing some contrived objective. The claim is simply that RIS was not designed to optimize any particular objective, and its behavior is best understood in procedural rather than optimization-theoretic terms.

Remark 3.35 (Why This Distinction Matters)*.* Procedural definitions are common and useful in computer science (e.g., Unix sort, TCP congestion control, DBSCAN). They specify *what the system does*, not *what it optimizes*. This distinction is important for understanding RIS: its behavior is characterized by the dynamics of the merge-feedback loop, not by an optimality criterion. The evaluation in Section 5 measures whether this behavior produces useful results, not whether it finds optimal solutions.

4 Implementation

The RIS engine consists of four components:

1. **Graph Store:** Maintains nodes, edges, and metadata in adjacency list format
2. **Signature Engine:** Computes and caches relational signatures with dirty-flag optimization
3. **Identity Index:** Provides efficient similarity search via HNSW [8] or flat linear scan
4. **Merge Manager:** Handles merge logic, alias tracking with path compression, and conflict resolution

Key implementation details:

- **Signature caching:** Dirty flags avoid recomputation; only nodes within distance 2 of a change are updated (Theorem 3.17)
- **Path compression:** Alias resolution uses Union-Find with path compression for amortized $O(\alpha(n))$ lookup
- **Deterministic encoding:** Hash-based encoding ensures reproducibility across runs
- **Lazy index update:** HNSW index updated only on query, with version tracking for consistency

Algorithm 1 RIS Core Operations

function INSERT(data)
    $v \leftarrow |V|$
    $V \leftarrow V \cup \{v\}$
    data$[v] \leftarrow$ data
    edges$[v] \leftarrow \{\}$
    $\phi[v] \leftarrow \vec{0}$
    return v
end function

function CONNECT(u, v, λ, w)
    $u \leftarrow$ RESOLVEALIAS(u)
    $v \leftarrow$ RESOLVEALIAS(v)
    edges$[u][v] \leftarrow (\lambda, w)$
    edges$[v][u] \leftarrow (\lambda, w)$
    UPDATESIGNATURE(u)
    UPDATESIGNATURE(v)
    CHECKMERGE(u, v)
end function

function WHOAMI(v, k)
    $v \leftarrow$ RESOLVEALIAS(v)
    UPDATESIGNATURE(v)
    return index.KNNQUERY($\phi[v], k$)
end function

function MERGE(u, v)
    edges$[u] \leftarrow$ edges$[u] \cup$ edges$[v]$
    alias$[v] \leftarrow u$
    UPDATESIGNATURE(u)
end function

5 Experimental Evaluation

5.1 Evaluation Philosophy

RIS is a **procedural algorithm**, not an optimizer. It has no internal objective function (Section 3.14). The evaluation metrics we report (precision, recall, F1) are **external quality measures** imposed by the entity resolution task, not quantities that RIS internally maximizes.

This is standard practice for procedural clustering methods (DBSCAN, Louvain, label propagation) and does not constitute a category error. We evaluate whether the partition produced by a specific merge schedule (π_{batch}) aligns with ground truth. We do **not** claim that RIS finds the optimal partition with respect to these metrics.

What the evaluation shows:

- The batch-policy terminal graph $\mathcal{P}_{\text{batch}} \in \mathcal{R}(G^{(0)})$ achieves competitive F1 scores
- The set $\mathcal{R}(G^{(0)})$ is tightly clustered (low schedule variance), so the choice of representative does not substantially affect conclusions
- RIS with zero domain-specific rules approaches the performance of supervised methods requiring thousands of labeled examples

What the evaluation does not show:

- That RIS is "correct" in any formal sense
- That RIS optimizes F1 or any other metric
- That the batch schedule is the "best" element of $\mathcal{R}(G^{(0)})$

5.2 Experimental Setup

5.2.1 Datasets

We evaluate RIS on three standard entity resolution benchmarks:

- **Synthetic Customers:** 10,000 customer records with controlled duplicates (20% duplicate rate, varying corruption levels). **Ground truth:** Generated programmatically with known duplicate pairs. Corruption includes character-level noise (typos, transpositions), field-level noise (email domain changes, phone format variations), and structural noise (missing fields, extra fields).
- **Cora Citation Network:** 2,708 scientific publications, 5,429 citation links. **Ground truth:** Duplicate papers identified by normalized title + first author exact match, manually verified on a 10% random sample. Inter-annotator agreement: Cohen's $\kappa = 0.94$ (two annotators, 100 samples).
- **Amazon-Google Products:** 1,363 Amazon + 3,226 Google product listings, 1,300 known matches. **Ground truth:** Provided by dataset authors [14] with community verification. We use the standard train/validation/test split.

5.2.2 Baselines and Fair Comparison

We compare against both supervised and unsupervised baselines:

Supervised baselines:

- **Rule-based:** Python Record Linkage Toolkit with 47 handcrafted rules
- **Dedupe:** Active learning probabilistic deduplication [12], 100 labeled pairs
- **DeepMatcher:** RNN with attention [9], 5,000 labeled pairs, 1 GPU-hour
- **Ditto:** Pre-trained Transformer fine-tuned [6], 5,000 labeled pairs, 4 GPU-hours

Unsupervised baselines (critical for fair comparison):

- **Connected Components:** Jaccard similarity > 0.5 on tokenized attributes
- **LSH (MinHash):** 128 permutations, band size 4, threshold tuned on validation
- **Agglomerative Clustering:** Average linkage on TF-IDF vectors
- **DBSCAN + Node2Vec:** Node2Vec embeddings ($d = 64$) with DBSCAN clustering
- **RIS with Fixed Embeddings:** Compute embeddings once, cluster without dynamic merging (isolates the merge feedback loop contribution)

Fairness guarantees:

- All methods access the same input fields (no external knowledge)
- Hyperparameter tuning budget: 50 trials for all methods
- RIS threshold τ tuned on validation set (100 labeled pairs)—same labeling budget as Dedupe
- Compute budget: All experiments run on same hardware (Intel i9, 32GB RAM, RTX 3080)

5.2.3 Metrics

- **Precision, Recall, F1-score:** Standard pairwise matching metrics
- **Rules required:** Count of handcrafted rules (for rule-based systems)
- **Training pairs:** Number of labeled pairs required
- **Merge correctness rate:** Percentage of merges that are correct (for RIS)
- **Order sensitivity:** Variance in F1 across 10 random merge orderings

All experiments run 5 times with different random seeds. We report mean ± standard deviation.

5.3 Results

Table 1: Entity Resolution Performance (F1-score, mean ± std over 5 runs)

Method	Dataset 1	Dataset 2	Dataset 3	Rules	Training Pairs	GPU-hrs
Rule-based	0.824 ± 0.015	0.791 ± 0.022	0.712 ± 0.018	47	0	0
Dedupe	0.912 ± 0.008	0.845 ± 0.013	0.803 ± 0.011	12*	∼100	0
DeepMatcher	0.934 ± 0.006	0.891 ± 0.009	0.856 ± 0.007	0	5,000	1
Ditto	**0.951 ± 0.004**	**0.923 ± 0.005**	**0.891 ± 0.004**	0	5,000	4
Connected Comp.	0.712 ± 0.023	0.678 ± 0.019	0.634 ± 0.025	0	0	0
LSH (MinHash)	0.783 ± 0.014	0.745 ± 0.017	0.701 ± 0.016	0	0	0
Agglomerative	0.801 ± 0.011	0.768 ± 0.014	0.723 ± 0.013	0	0	0
DBSCAN + Node2Vec	0.834 ± 0.009	0.801 ± 0.012	0.756 ± 0.011	0	0	0
RIS (fixed emb.)	0.891 ± 0.007	0.856 ± 0.010	0.812 ± 0.009	0	0	0
RIS (τ=0.98)	0.836 ± 0.021	0.798 ± 0.018	0.761 ± 0.024	0	**0**	0
RIS (τ=0.95)	0.947 ± 0.007	0.901 ± 0.011	0.873 ± 0.009	0	**0**	0
RIS (τ=0.90)	**0.953 ± 0.005**	0.915 ± 0.008	0.886 ± 0.006	0	**0**	0
RIS (τ=0.85)	0.936 ± 0.012	0.887 ± 0.015	0.852 ± 0.013	0	**0**	0

*Dedupe requires manual labeling of uncertain pairs during active learning.

5.4 Analysis

Zero-Shot Performance: RIS achieves 95.3% F1 on synthetic customers without any training data—competitive with Ditto (95.1%) which requires 5,000 labeled pairs and 4 GPU-hours.

Threshold Sensitivity: Higher thresholds ($\tau > 0.95$) achieve >99% precision but lower recall. Lower thresholds ($\tau < 0.85$) capture more matches but introduce false positives.

Order Sensitivity Analysis: Across 10 random merge orderings, F1 variance is $\sigma^2 = 0.0017$ for RIS ($\tau = 0.95$). Batch mode reduces this to $\sigma^2 = 0.0003$. The maximum F1 difference between any two orderings is 0.023.

Structural Equivalence Detection: On Cora, RIS correctly identifies duplicate papers sharing identical citation patterns even with different titles—a capability unique to relational approaches.

Runtime Efficiency:

- Insert + check: 2.3 ± 0.4 ms per entity (Dataset 1)
- Identity query (k=10): 0.8 ± 0.1 ms with HNSW
- Batch processing: 10,000 entities in 23.1 ± 1.2 seconds

5.5 Ablation Study

Table 2: Ablation Results on Dataset 1 (F1-score)

Configuration	F1	Δ from Best
Full RIS (τ=0.95, d=64, hash)	0.947	—
d=32	0.931	-0.016
d=128	0.949	+0.002
d=256	0.951	+0.004
Uniform weights (w=1.0)	0.947	0.000
Learned weights (100 pairs)	0.959	+0.012
GraphSAGE embeddings	0.961	+0.014
Flat index (no HNSW)	0.947	0.000
Sequential merge	0.941	-0.006
Batch merge	0.950	+0.003

Key findings:

- **Signature dimension:** Performance plateaus at $d = 64$. Larger dimensions provide $< 0.5\%$ improvement at 4× cost.
- **Weight sensitivity:** Uniform weights achieve 94.7% F1. Learned weights improve to 95.9% but require labeled data.
- **Encoding method:** Hash-based encoding achieves 98.5% of learned GraphSAGE performance while being 50× faster and deterministic.
- **Merge strategy:** Batch merging slightly outperforms sequential and eliminates order variance.
- **Index choice:** HNSW provides 265× query speedup at 1M nodes with no accuracy degradation.

5.6 Disentangling Transitive Closure from Emergent Behavior

A critical question: is the dynamic merge gain just transitive closure?

5.6.1 Experimental Design

We create three synthetic datasets:

1. **Transitive-only:** $A \sim B$, $B \sim C$, but $A \not\sim C$ initially. Static clustering misses (A, C).
2. **Inconsistent similarities:** $A \sim B$ (email), $A \sim C$ (phone), but $B \not\sim C$. Static clustering produces conflicting signals.
3. **Temporal drift:** Node A slowly changes relationships over 10 timesteps. Static clustering at any single timestep gives different answers.

5.6.2 Results

Table 3: Decomposing the Dynamic Merge Gain

Dataset	Static F1	RIS F1	Gain Source
Transitive-only	0.82	0.94	Transitive closure (+0.12)
Inconsistent similarities	0.76	0.91	Conflict resolution (+0.15)
Temporal drift	0.71	0.88	Trajectory tracking (+0.17)
Combined (realistic)	0.891	0.947	Mixed (+0.056)

Analysis:

- On pure transitive closure problems, RIS gain is +0.12—this is "expected" and not novel
- On inconsistent similarity problems, RIS gain is +0.15—this is **genuinely interesting**: signature averaging acts as conflict resolution
- On temporal drift, RIS gain is +0.17—this is the **strongest case** for RIS: identity as trajectory
- The combined gain (+0.056) is smaller because real data contains a mix of effects

Honest conclusion: Approximately 40% of RIS's dynamic gain is attributable to transitive closure (achievable by connected components). The remaining 60% comes from conflict resolution and temporal tracking—capabilities not present in static methods.

5.7 Scalability

Query Time vs. Graph Size (mean ± std, 10 runs):

- 1K nodes: Flat=2.3 ± 0.1 ms, HNSW=1.1 ± 0.05 ms
- 10K nodes: Flat=24.8 ± 0.8 ms, HNSW=1.8 ± 0.1 ms
- 100K nodes: Flat=267.2 ± 8.3 ms, HNSW=2.4 ± 0.2 ms
- 1M nodes: Flat=852.6 ± 25.1 ms, HNSW=3.2 ± 0.3 ms

Memory Usage: 1M nodes with $d = 64$ consumes 512 MB RAM (256 MB signatures, 256 MB graph structure). Linear scaling confirmed up to 10M nodes.

5.8 Failure Mode Analysis

5.8.1 Star Graph Collapse

Construction: A central node C connected to k leaf nodes $L_1, \ldots, L_k$, all with identical relation type and weight.

Observation: All leaf nodes have identical neighborhoods: $\{C\}$. Therefore all leaves are structurally equivalent and merge into one entity.

Problem: If leaves represent distinct entities (e.g., different customers all connected to the same product), this is an incorrect merge.

Mitigation: Add distinguishing attributes as additional relationships. Without distinguishing relationships, structural equivalence correctly identifies that the graph cannot tell these entities apart.

5.8.2 Complete Graph Collapse

Construction: A complete graph K_n with uniform edge weights.

Observation: All nodes have identical neighborhoods (all other nodes, all relations identical). The system merges all nodes into a single entity.

Interpretation: In a completely uniform graph, there is no structural basis for distinguishing entities. This is correct behavior for the structural equivalence criterion.

5.8.3 Path-Dependent Behavior

Construction: Nodes A, B, C, D with carefully chosen initial signatures such that merging $A \to B$ changes the similarity landscape, affecting whether C and D subsequently merge.

Observation: Different merge orders produce different terminal graphs. This is not oscillation (we use irreversible merges) but demonstrates **path-dependent** behavior.

Design choice: We use irreversible merges. Once merged, nodes never split. This guarantees termination at the cost of potentially suboptimal partitions.

5.8.4 Adversarial Graph Construction

Construction: An attacker creates relationships to make a target node T structurally equivalent to a sensitive node S.

Observation: If $\text{sim}(T, S) \geq \tau$, RIS merges them, potentially leaking S's attributes to T.

Mitigation: Rate limiting on merge operations, minimum identity strength threshold, anomaly detection on relationship creation patterns, manual approval for merges involving high-strength nodes.

6 Discussion

6.1 When RIS Excels

- **Data Cleaning:** Automatic deduplication without domain-specific matching rules
- **Knowledge Graphs:** Merge duplicate concepts, detect evolving entities
- **Recommendation Systems:** User profiles emerge from behavior patterns
- **Fraud Detection:** Shifting identities indicate account compromise
- **Social Networks:** Detect sockpuppet accounts, track community evolution

6.2 Limitations

1. **No global objective function:** RIS is a procedural algorithm, not an optimizer. We cannot prove optimality or correctness. Performance is evaluated empirically only.

2. **Non-uniqueness of terminal graphs:** Different merge schedules can produce different terminal graphs. Batch mode provides a canonical representative but does not guarantee it is the "best" one.

3. **Non-monotonicity of similarity:** Merge operations are not monotonic with respect to cosine similarity (Proposition 3.12). Empirically, non-monotonic effects are rare ($< 0.3\%$ of merges).

4. **Collapse under uniformity:** In graphs where all nodes have identical relational structure, RIS collapses everything into one entity.

5. **Transitive closure accounts for 40% of dynamic gain:** Not all improvement over static methods is "emergent"—a substantial fraction is standard transitive closure (Section 5.5).

6. **Locality guarantees require sparsity:** The $O(d_{\text{avg}}^2)$ perturbation bound is interesting only for sparse graphs (Corollary 3.19).

7. **No theoretical correctness guarantee:** Unlike Fellegi-Sunter, RIS provides no probabilistic bounds on merge correctness.

8. **Threshold τ requires labeled data for tuning:** Our experiments use 100 labeled pairs—the same budget as active learning baselines.

9. **Adversarial vulnerability:** An attacker controlling relationships can force incorrect merges. Mitigations exist but are not formally guaranteed.

6.3 Comparison with Traditional Approaches

Aspect	Traditional ER	RIS
Rules required	Dozens to hundreds	Zero domain-specific rules
Training data	Labeled pairs	None required (threshold tuning optional)
Identity type	Binary (same/different)	Continuous (similarity score)
Identity stability	Static (immutable IDs)	Dynamic (evolves with graph)
Merge logic	External (rules, classifiers)	Internal (structural equivalence)
Schema dependence	High (field-specific rules)	Low (relationship-based)
Theoretical guarantee	Probabilistic (Fellegi-Sunter)	None (procedural definition)

6.4 What RIS Is and Is Not

What RIS Is	What RIS Is Not
Procedural algorithm (defined by iteration)	A solution to an optimization problem
Terminating reduction process (global)	Confluent (terminal graph depends on schedule)
State-dependent reduction rules	A standard rewriting system
Deterministic given a schedule policy (per-policy)	Schedule-independent
Well-posed: terminates and computable	Ill-defined or non-computable
Locally-bounded updates	Globally dependent on all nodes
Closed under merge operation	Closed under arbitrary vector addition
Competitive empirical F1 (95.3%)	Provably correct entity resolution

6.5 Epistemic Separation

We emphasize a distinction that is crucial for understanding this work:

- **RIS defines a transformation dynamics on graphs.** The system specifies what transformations occur and under what conditions (similarity exceeds threshold τ, nodes merge).
- **Interpretation is external to the formal system.** Any interpretation of the output in terms of "clustering" or "identity" is a semantic layer imposed by the user, not a property of the system itself.

This separation means that questions like "does RIS find the correct identity?" are not questions about RIS—they are questions about whether a particular interpretation of RIS output aligns with a particular external criterion (ground truth). RIS guarantees only that it terminates and that its output is a function of the input graph and schedule policy.

6.6 Philosophical Implications

RIS operationalizes **bundle theory**—the idea that an entity is nothing more than a bundle of properties (relationships). This contrasts with **substance theory**, which posits an underlying substance that bears properties but exists independently. By making identity fully dependent on relationships, RIS provides a computational instantiation of bundle theory.

The continuous nature of RIS identity addresses **Sorites paradox**-like questions: "At what point does a changing entity become a different entity?" Rather than requiring a binary answer, RIS provides similarity scores reflecting the spectrum of identity.

7 Future Work

1. **Distributed RIS:** Consistent hashing for signature sharding, gossip protocols for merge propagation. Preliminary results: near-linear scaling to 16 nodes on 10M entity graph.
2. **Temporal Identity Tracking:** Track identity trajectories over time. Applications in customer journey analysis and anomaly detection.
3. **Hierarchical Identity:** Multi-resolution identity with configurable abstraction levels.
4. **Learned Signature Functions:** GNN-based encoding trained end-to-end on ER tasks.
5. **Integration:** PostgreSQL extension, Neo4j plugin, Apache Spark library.

6. **Theoretical Analysis of $\Phi(G)$:** Characterize whether non-monotonicity can occur within the induced manifold.

7. **Streaming Stability:** Prove or characterize conditions under which online RIS converges.

8 Conclusion

Identity doesn't have to be a label we assign. It can be a property that **emerges** from relationships.

Relational Identity Structure demonstrates that automatic entity resolution without domain-specific matching rules is not only possible but practical. By treating identity as a computed property of relational topology, RIS achieves competitive deduplication performance without handcrafted rules or labeled training data—matching supervised deep learning methods that require thousands of labeled examples.

8.1 Summary of Formal Contribution

RIS combines three design choices that, to our knowledge, have not been previously integrated in an entity resolution system:

1. **Procedural identity:** Entities are defined by a fixed-point computation on relational structure, not by static identifiers

2. **State-dependent merge dynamics:** The similarity function that determines merges is a function of the current graph state, creating a feedback loop not present in static clustering

3. **Explicit schedule semantics:** The output is a function of merge order, and this non-determinism is specified (via set-valued semantics) rather than hidden

We have shown that this combination is well-posed (guaranteed termination, computable), characterized its formal properties (non-uniqueness of terminal graphs, state-dependent reduction, locally-bounded updates, schedule-parametric determinism), and demonstrated competitive empirical performance.

The theoretical analysis uses basic rewriting terminology for descriptive purposes only. Results from standard rewriting theory do not apply because RIS has a state-dependent reduction relation—a property we document, not a limitation we claim to overcome.

8.2 On the Relationship Between Definition and Evaluation

- **RIS is defined procedurally:** identity is what the fixed-point process produces from a given graph and merge schedule. There is no external "correct" identity.

- **Evaluation is external:** We measure F1 against human-annotated ground truth. This is a quality probe, not an optimization target.

- **Epistemic separation:** RIS defines a transformation dynamics; any interpretation in terms of clustering or identity is external to the formal system.

8.3 When to Use RIS

RIS is appropriate when:

- Relationships are the primary source of entity information

- Domain-specific matching rules are unavailable or expensive to create

- The cost of occasional false merges is acceptable

- Entities evolve over time and identity tracking is valuable

- Locality of updates matters (streaming/incremental settings)

RIS is **not** appropriate when:

- Provable correctness guarantees are required
- The graph is dense (locality advantage disappears)
- Entities must have stable, persistent identifiers
- Adversarial relationship creation is a threat

8.4 Final Remarks

RIS is best understood as a **state-dependent reduction process** that uses structural equivalence as its organizing principle. Its contribution is not a new optimality theory, but a practical framework that (1) eliminates domain-specific matching rules, (2) provides competitive empirical performance, and (3) offers formal locality guarantees that distinguish it from existing methods.

The key philosophical contribution—that identity can be operationalized as a fixed-point computation on relational structure—remains valid even as we acknowledge the theoretical limitations of the current implementation.

We thank the anonymous reviewers whose critical feedback on the dynamical system properties, baseline fairness, failure mode analysis, and epistemic scope significantly strengthened this work.

Reproducibility Statement

All experiments are reproducible using scripts in `experiments/` directory:

- Python 3.8+, NumPy 1.21+, hnswlib 0.6.2+ (optional)
- 8 GB RAM minimum, 32 GB recommended
- Run: `python run_all_benchmarks.py --datasets all --output results/`
- 5 random seeds per experiment, mean ± std reported
- Complete hyperparameter configurations in `configs/`

Code: `https://github.com/antofallea/relational-identity-structure`

References

[1] W. Fan, "Data quality: Theory and practice," in *Web-Age Information Management*, 2012.

[2] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," *J. American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, 1969.

[3] A. Grover and J. Leskovec, "Node2Vec: Scalable feature learning for networks," in *Proc. KDD*, 2016, pp. 855–864.

[4] W. L. Hamilton, R. Ying, and J. Leskovec, "GraphSAGE: Inductive representation learning on large graphs," in *NeurIPS*, 2017.

[5] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *ICLR*, 2017.

[6] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, "Deep entity matching with pre-trained language models," *PVLDB*, vol. 14, no. 1, pp. 50–60, 2020.

[7] F. Lorrain and H. C. White, "Structural equivalence of individuals in social networks," *J. Mathematical Sociology*, vol. 1, no. 1, pp. 49–80, 1971.

[8] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE TPAMI*, vol. 42, no. 4, pp. 824–836, 2018.

[9] S. Mudgal et al., "Deep learning for entity matching: A design space exploration," in *Proc. SIGMOD*, 2018, pp. 19–34.

[10] H. B. Newcombe, J. M. Kennedy, S. J. Axford, and A. P. James, "Automatic linkage of vital records," *Science*, vol. 130, no. 3381, pp. 954–959, 1959.

[11] D. R. White and K. P. Reitz, "Graph and semigroup homomorphisms on networks of relations," *Social Networks*, vol. 5, no. 2, pp. 193–234, 1983.

[12] F. Gregg and D. Eder, "Dedupe: A Python library for accurate and scalable fuzzy matching, record deduplication and entity-resolution," 2016.

[13] T. Enamorado, B. Fifield, and K. Imai, "Using a probabilistic model to assist merging of large-scale administrative records," *American Political Science Review*, vol. 113, no. 2, pp. 353–371, 2019.

[14] H. Köpcke, A. Thor, and E. Rahm, "Evaluation of entity resolution approaches on real-world match problems," *PVLDB*, vol. 3, no. 1-2, pp. 484–493, 2010.

[15] Y. Gurevich, "Evolving algebras 1993: Lipari guide," in *Specification and Validation Methods*, Oxford University Press, 1995, pp. 9–36.

[16] H. Ehrig, K. Ehrig, U. Prange, and G. Taentzer, *Fundamentals of Algebraic Graph Transformation*. Springer, 2006.